

High-Level Design (HLD): RAG-Based Customer Support Assistant

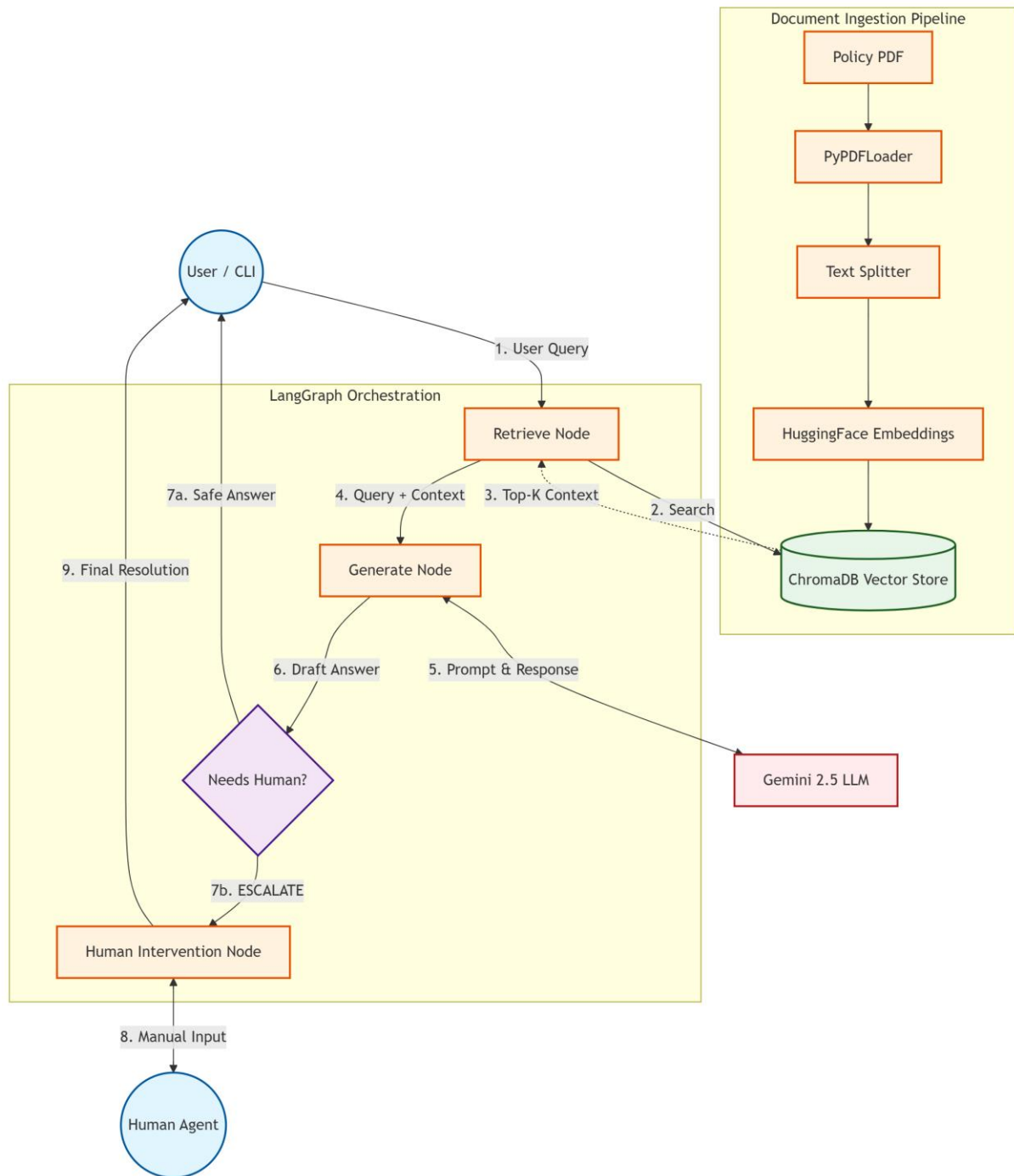
1. System Overview

Problem Definition

Modern customer support centers face overwhelmingly high volumes of repetitive queries regarding return policies, product guidelines, and basic troubleshooting. While standard Large Language Models (LLMs) can converse naturally, they lack access to proprietary company data and are prone to hallucinating policies. Conversely, relying entirely on human agents for basic knowledge retrieval is inefficient, costly, and unscalable.

Scope of the System This project implements a Retrieval-Augmented Generation (RAG) system grounded exclusively in official, approved policy documents to ensure factual accuracy. It utilizes a stateful, graph-based orchestration workflow to intelligently route user queries. The system automatically resolves standard policy questions while enforcing a strict Human-in-the-Loop (HITL) escalation protocol for complex, out-of-bounds, or low-confidence scenarios, seamlessly transferring context to a human agent.

2. Architecture Diagram



3. Component Description

- **Document Loader (PyPDFLoader):** Handles the ingestion and text extraction of the raw PDF knowledge base (e.g., Return Policies, FAQs).
- **Chunking Strategy (RecursiveCharacterTextSplitter):** Breaks the extracted document into manageable 500-character chunks with a 50-character overlap. This

overlap is crucial for preserving semantic context and preventing sentences from being cut in half.

- **Embedding Model (HuggingFace all-MiniLM-L6-v2):** A local embedding model that converts the text chunks into dense, 384-dimensional mathematical vectors, capturing the semantic meaning of the text.
- **Vector Store (ChromaDB):** Acts as the persistent local database to store the embeddings and execute rapid similarity searches against user queries.
- **Retriever:** The search mechanism that executes a similarity search ($k=3$) to fetch the top 3 most contextually relevant document chunks from ChromaDB based on the user's input.
- **LLM (Google Gemini):** Serves as the reasoning engine. It synthesizes the retrieved chunks to draft a coherent answer or intelligently decides if the query violates its confidence threshold.
- **Graph Workflow Engine (LangGraph):** The state machine that orchestrates the application. It maintains the conversation state and executes conditional logic to transition between nodes (Retrieval $\rightarrow$ Generation $\rightarrow$ Routing).
- **Routing Layer & HITL Module:** Evaluates the LLM's output. If the LLM produces a confident answer, it routes to the user. If the LLM outputs an escalation trigger, it pauses the graph execution, alerts the terminal, and waits for a human agent to input a manual override response.

4. Data Flow

Document Ingestion Pipeline (Offline Phase):

1. A raw policy PDF is loaded into the system.
2. The text is extracted and split into semantic chunks.
3. The local embedding model translates these chunks into vector embeddings.
4. The vectors are permanently indexed and stored in the ChromaDB vector database.

Query Lifecycle & Routing (Runtime Phase):

1. The user submits a support query via the Command Line Interface (CLI).
2. The query is embedded, and the `Retrieve Node` searches ChromaDB for the closest vector matches, returning the top-K relevant text chunks.
3. The `Generate Node` passes both the user's original query and the retrieved text context to the LLM via a strict prompt.
4. **Decision & Routing:**
 - *Path A (Resolved):* The LLM finds the answer in the context, drafts a response, and the system delivers it to the user.
 - *Path B (Escalated):* The LLM determines the query is complex, missing from the context, or out-of-bounds. It triggers an "ESCALATE" flag.
5. **HITL Flow:** The LangGraph orchestrator intercepts the escalation flag, suspends execution, and triggers the `Human Intervention Node`. A human agent reviews the query and provides manual input. The graph resumes and delivers the human's final resolution to the user.

5. Technology Choices

- **Vector Database (ChromaDB):** Selected because it provides seamless, lightweight local storage. It does not require cloud infrastructure or paid API keys, making it highly optimal for rapid prototyping and secure local development.
- **Orchestration (LangGraph):** Standard linear chains (like LangChain's basic LCEL) cannot easily handle cyclical loops or paused states. LangGraph was chosen specifically for its stateful architecture, allowing the system to literally pause execution, wait for external human input, and resume without losing context.
- **LLM (Google Gemini):** Chosen for its exceptional inference speed and reasoning capabilities. Speed is critical during the generation and routing nodes to ensure the user does not experience high latency while waiting for the system to decide whether to answer or escalate.
- **Embeddings (HuggingFace Local):** By utilizing the open-source `sentence-transformers` library locally, the system completely avoids the network latency, API costs, and rate limits associated with cloud-based embedding models.

6. Scalability Considerations

- **Handling Large Documents:** The defined chunking strategy ensures that even if a 500-page operational manual is ingested, the system will only ever pass the top 3 most relevant paragraphs to the LLM. This strictly controls token usage and prevents exceeding the LLM's context window.
- **Increasing Query Load:** While currently operating via a CLI, the LangGraph architecture is entirely decoupled from the frontend. To scale for thousands of users, the `app.py` script can be replaced with an asynchronous FastAPI backend, instantly turning the workflow into a highly concurrent web service.
- **Latency Optimization:** Latency is minimized by keeping the heaviest data components (the vector database and the embedding models) entirely on local hardware. The only required network call during a standard query is the final generation step to the LLM API.